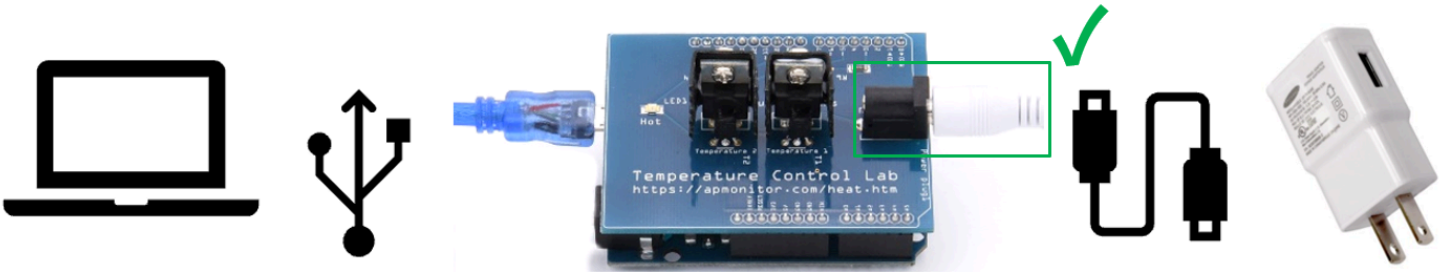
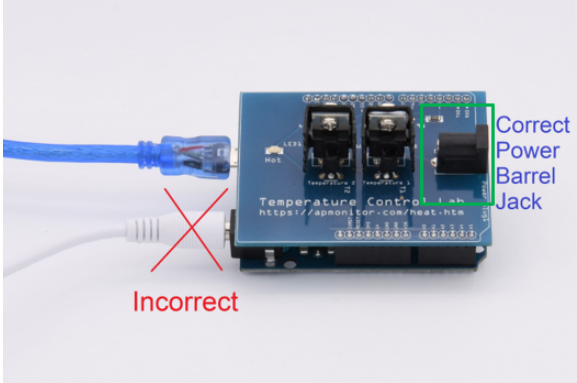
Para esta guía se debe de haber completado la instalación previa y prueba LED con el TCLab.

##

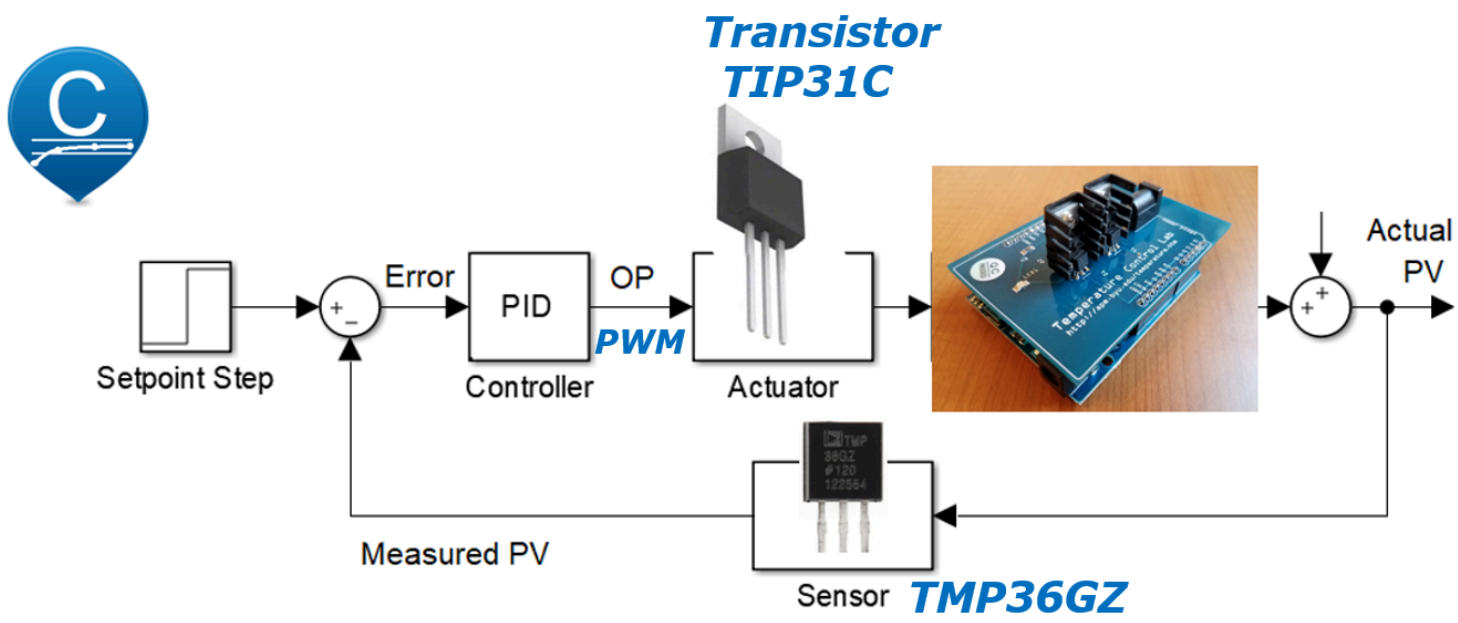
1) Iniciamos con la conexión del USB del Arduino a la computadora y a la fuente de alimentación siguiendo el siguiente esquema:



nota: **no** se puede conectar de otra forma (ej alimentación al terminal barril del Arduino) ya que no se tendrá la potencia suficiente para alimentar el TCLab y hacer las pruebas, como en la siguiente imagen estaría incorrecto:

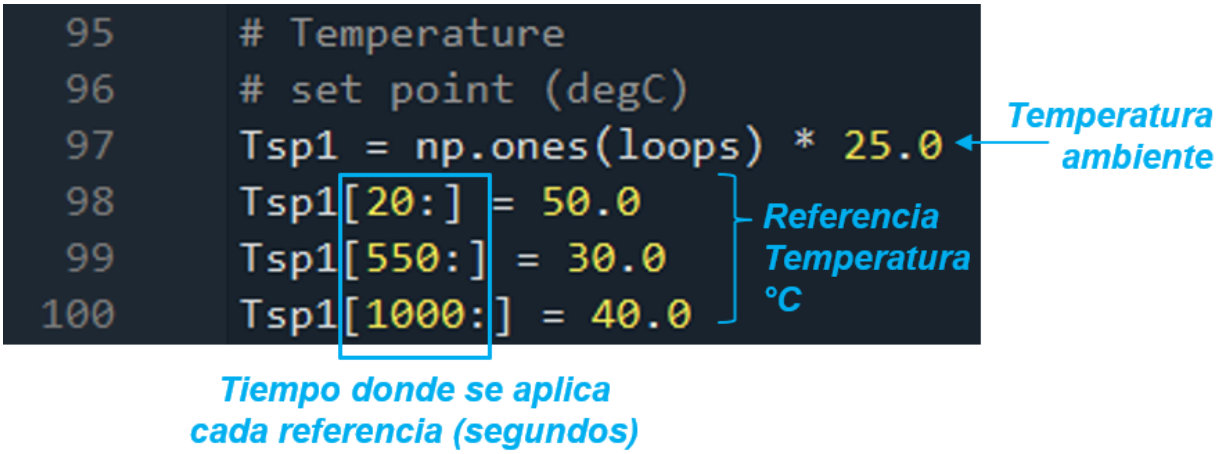


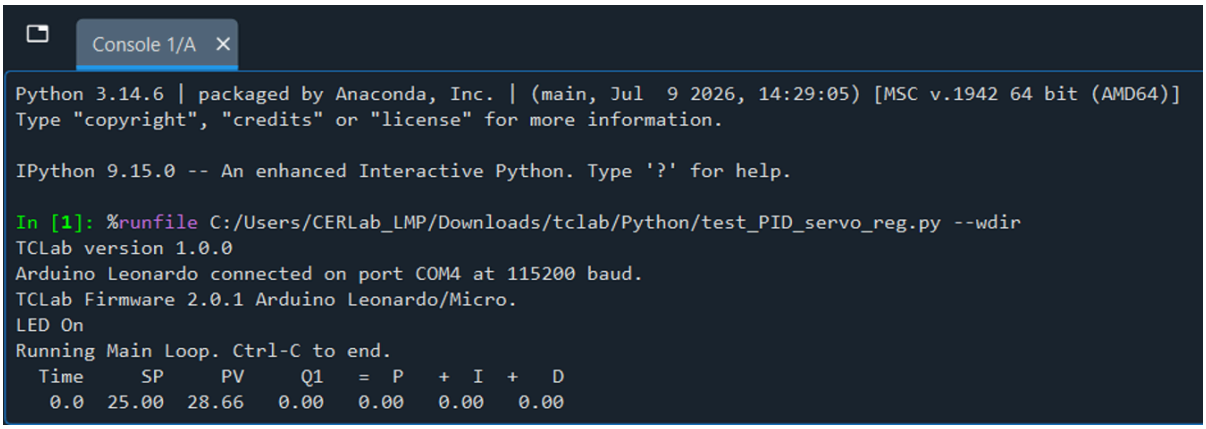
El lazo de control PID implementado en el Arduino empleará el **Transistor 1 (T1)** para modificar la temperatura del sistema al valor de la referencia, a la vez que rechaza las posibles perturbaciones que se apliquen al sistema:



##########

2) Descargamos el archivo con el código Python para la ejecución de la prueba en lazo cerrado ***test_PID_servo_reg.py*** (carpeta Datos de Temperatura, sección Proyecto de Mediación Virtual) en la carpeta de descargas, y lo **abrimos** con el IDE Spyder que emplearemos para la ejecución de los códigos, el cual se instaló junto con Anaconda (debe ser el instalado en Anaconda, se puede buscar en el Anaconda Navigator en caso requerido)





##########

3 vez abierto el archivo ***test_PID_servo_reg.py*** en el Spyder IDE, editamos los ***parámetros*** del controlador PID, sustituyendo los que obtuvimos del método de sintonización de controladores PID que se desea probar:

```
32  def pid(sp,pv,pv_last,ierr,dt):
33      Kc   = 10.0 # K/%Heater
34      tauI = 50.0 # sec
35      tauD = 1.0  # sec
```

PID (estándar)

##

4) Editamos a conveniencia el **tiempo** total de ejecución, se recomienda para 3 escalones en la referencia y 2 perturbaciones, un tiempo de al menos 35min:

```
88  # Run time in minutes
89  run_time = 35.0
```

##

5) Editamos los tiempos donde se aplicarán los cambios en la ***referencia***, así como los valores del punto de operación asignado para cada equipo como las referencias en °C a seguir por el sistema en cada cambio:

```
 95  # Temperature
 96  # set point (degC)
 97  Tsp1 = np.ones(loops) * 25.0
 98  Tsp1[20:] = 50.0
 99  Tsp1[550:] = 30.0
100  Tsp1[1000:] = 40.0
```

Temperatura ambiente

Referencia Temperatura °C

Tiempo donde se aplica cada referencia (segundos)

##

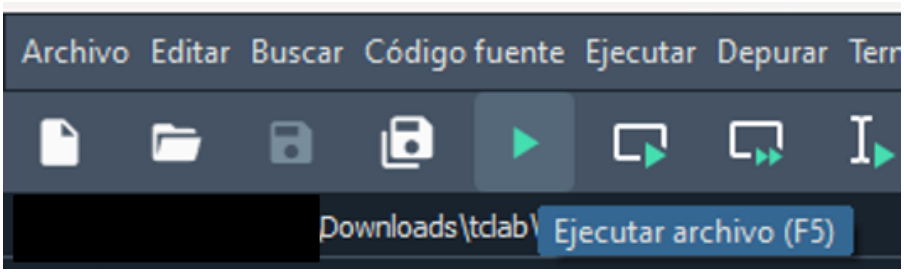
6) Verificamos la aplicación de la ***perturbación 1*** al calentar el ambiente mediante el transistor 2 mediante las siguientes líneas. Se pueden editar los tiempos en caso requerido, aunque se recomienda probar con la configuración suministrada (encendido en 1500s, apagado en 1800s). Si se va a modificar, se recomienda dejar 500s o más desde la 3era referencia para aplicar la perturbación, y al menos 300s para que esta sea rechazada por el controlador.

```
110  # disturbance
111  Q2[1500:] = 100.0
112  Q2[1800:] = 0.0
```

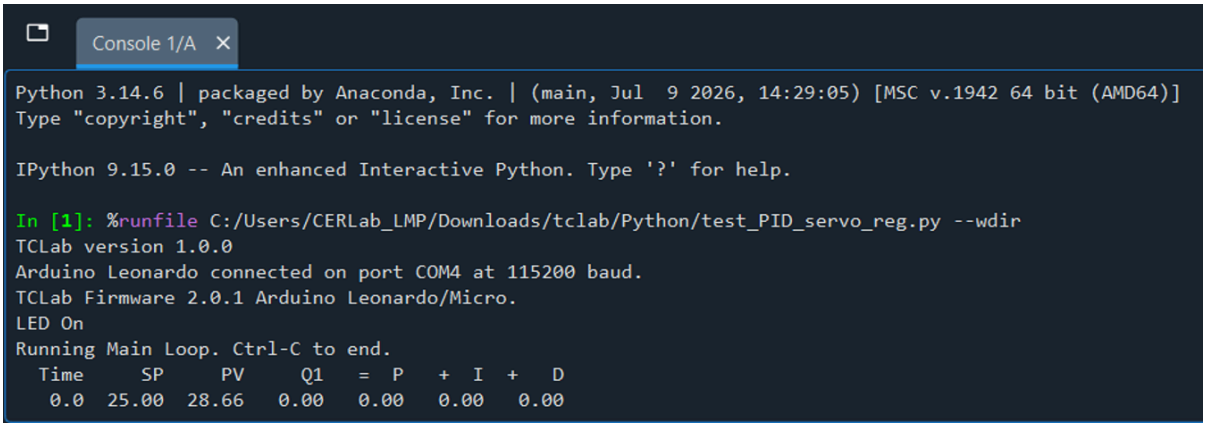
Debe verificarse que se apaga el transistor 2 de la perturbación 1 antes de aplicar cualquier otra perturbación al sistema, para no sobrecargar el sistema por el consumo de corriente de ambos transistores operando de forma simultánea. Esto puede hacer que se apague por seguridad del TCLab, perdiendo todos los datos de la prueba en curso.

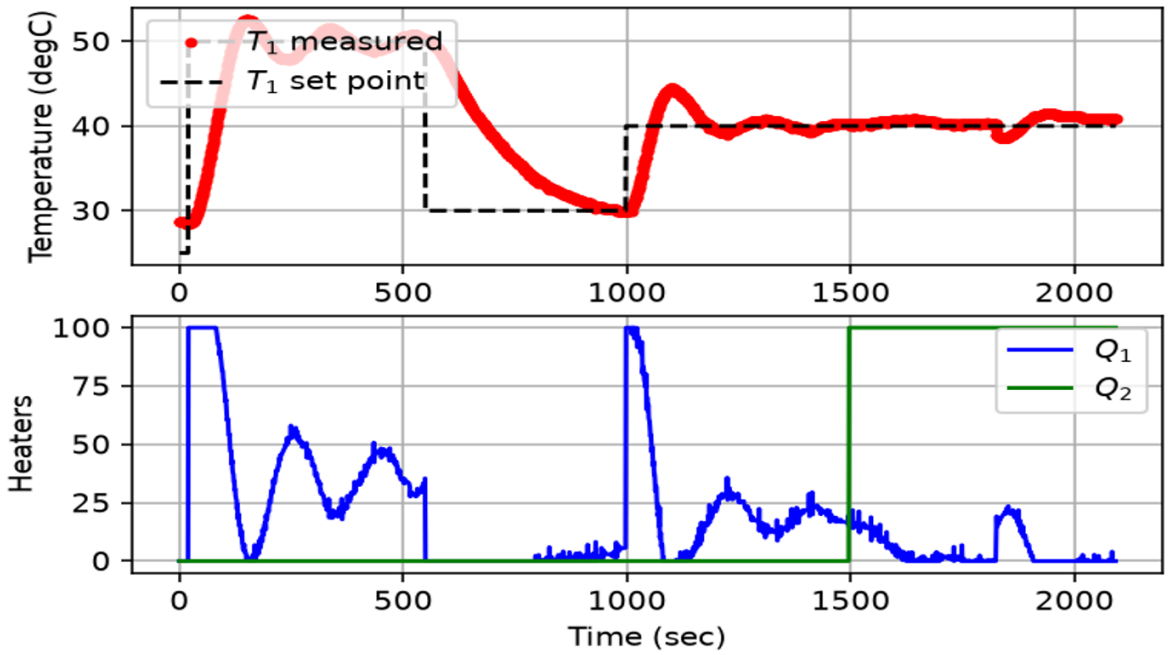
##

7) Ejecutamos el código ***test_PID_servo_reg.py*** en el Spyder IDE (botón play / Run File)



El algoritmo de control empieza a ejecutarse para realizar las pruebas y generar las gráficas:

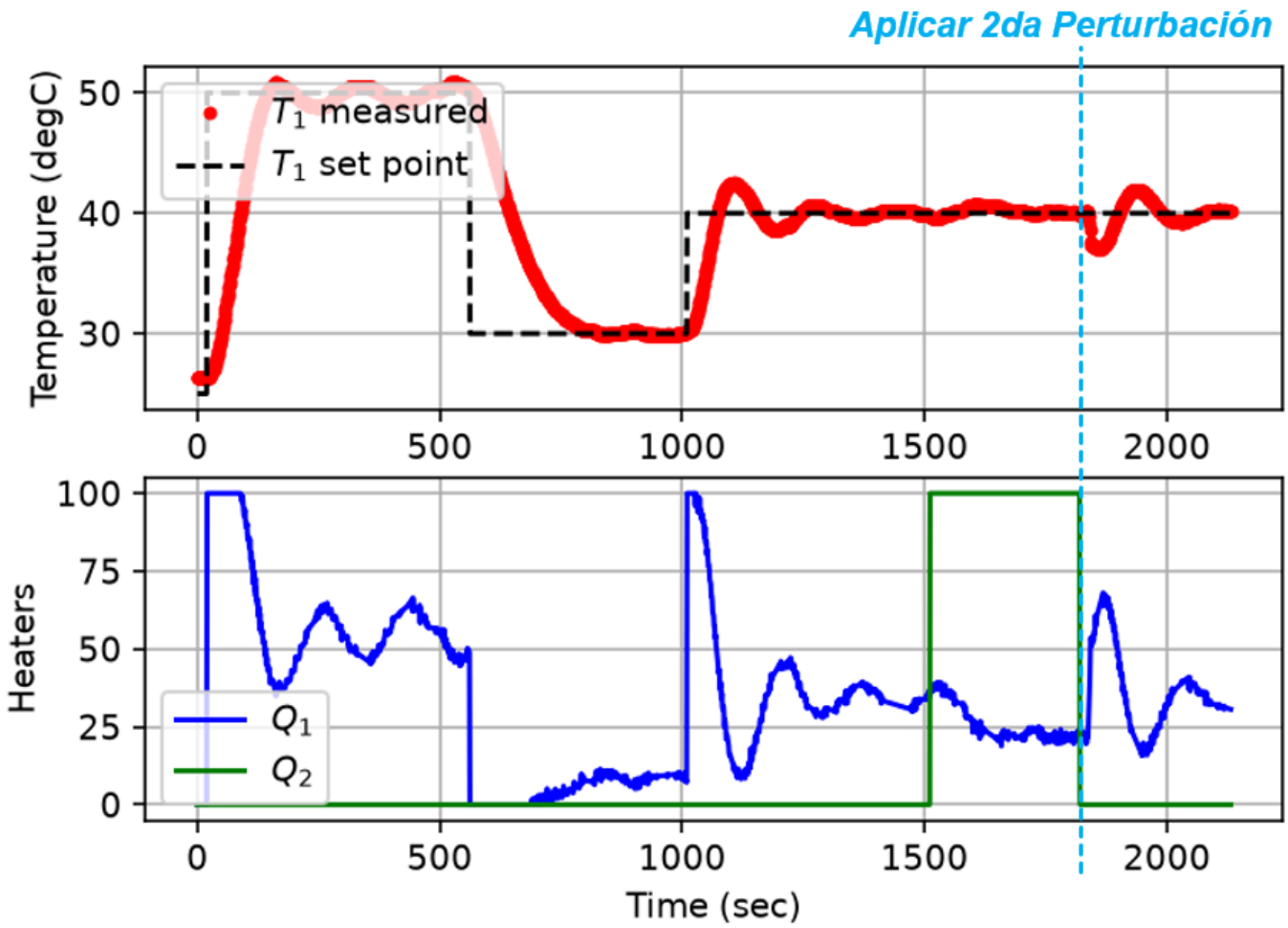


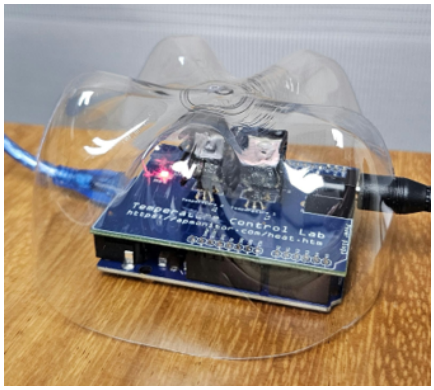


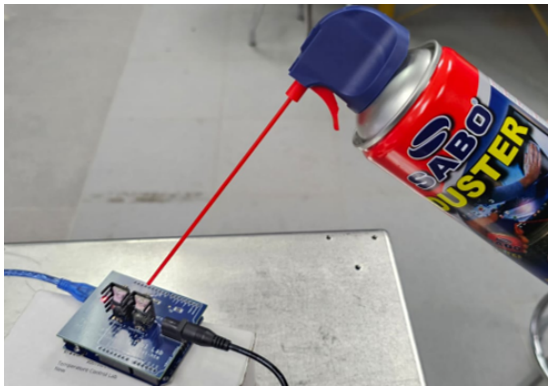
##

8) Para la ***segunda perturbación***, utilizamos el implemento para "***soplar***" el transistor, o bien encerramos el TCLab en una cobertura plástica.

Se recomienda aplicarla en el último escalón de prueba, luego de apagar primera perturbación del calentador 2, en un tiempo superior a los 1801segundos (***anotar el tiempo de aplicación***), con el fin de no tener consumiendo el calentador 2 de forma simultánea al calentador 1, lo cual requiere una potencias elevada, que a su vez puede hacer que el TCLab se apague por seguridad, perdiendo los datos de la prueba.







##

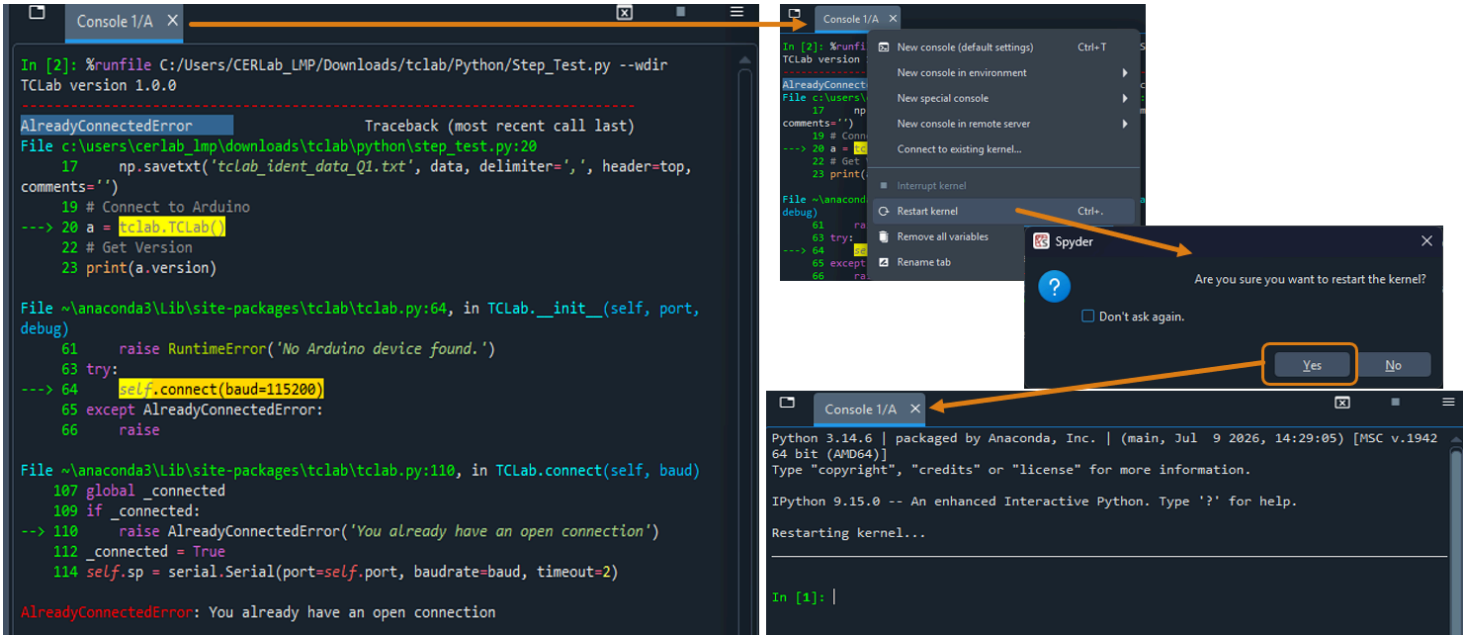
9) ***Al finalizar*** la ejecución correcta de la prueba se genera un archivo **data.txt** con los datos de la prueba experimental en la misma carpeta donde se tiene el archivo ***test_PID_servo_reg.py*** (normalmente carpeta de *descargas*). **Este archivo se sobrescribe en cada prueba, por lo que debe copiarse a otra ubicación antes de realizar otra prueba.**

Si se desea guardar copia del gráfico generado en el Spyder, dar click derecho y seleccionar la opción “save plot as…”



##

10) En caso del error: "*AlreadyConnectedError: You already have an open connection*" necesitaremos reiniciar el núcleo (kernel) de Python con click derecho en la terminal del Spyder IDE antes de volver a ejecutar la prueba:



Última modificación: martes, 27 de octubre de 2026, 02:08

© 2026 Universidad de Costa Rica - Tel. (506) 2511-5015

Consultas didácticas:
consultas.metics@ucr.ac.cr

Consultas técnicas:
soporte.metics@ucr.ac.cr

Dirección:
unidad.metics@ucr.ac.cr